Lab04

Max Stróżyk

Winter 2024/2025

English translation of the Polish original. Historical results and formulas retained.

Contents

1 Task 1 **2**
1.1 Overview of Horner's method . . . 2
1.2 Results and conclusions . . . 2

2 Task 2 **3**
2.1 Overview of Neville's algorithm . . . 3
2.2 Results and conclusions . . . 3

3 Task 3/4 **5**
3.1 Overview of Newton's algorithm . . . 5
3.2 Results and conclusions . . . 5

4 Task 5 **8**
4.1 Overview of Clenshaw's algorithm . . . 8
4.2 Results and conclusions . . . 8

5 Task 6 **9**
5.1 Overview of trigonometric interpolation . . . 9
5.2 Results and conclusions . . . 9

Project on GitHub

1 Task 1

1.1 Overview of Horner's method

Horner's method efficiently evaluates a polynomial $P(x)$ at a given point x. Its main advantage is to reduce the required operations, lowering computational complexity from $O(n^2)$ to $O(n)$, where n is the polynomial degree.

The algorithm's basic idea can be expressed recursively as follows:

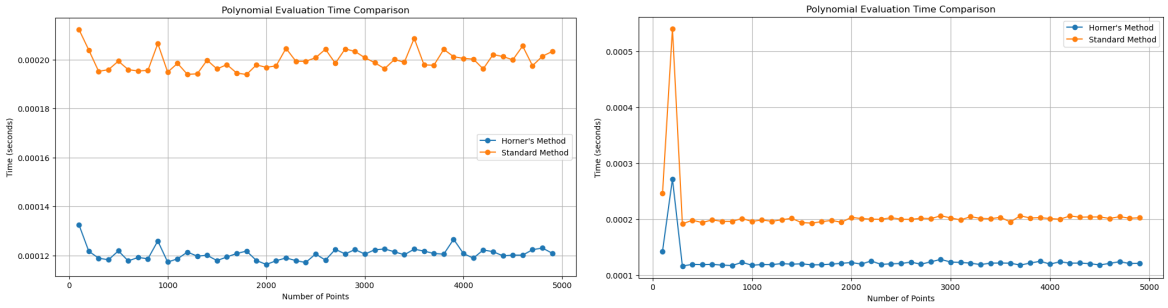
$$P(x) = a_n + x(a_{n-1} + x(a_{n-2} + \cdots + x(a_1 + xa_0)))$$

Evaluation requires only n multiplications and n additions, substantially reducing computation time for high-degree polynomials.

Tests used polynomial sizes from 100 to 5000 in steps of 100. For each degree, 100 different polynomials were sampled. Each polynomial was evaluated using Horner's method and the standard method; the measured runtimes were then averaged.

1.2 Results and conclusions

The two types of plots below illustrate the experimental results:



Conclusions:

- **Speed improvement:** Horner's method gives an average speed improvement of 1.64 ± 0.04, consistent with expectations from reduced computational complexity.
- **Unexpected runtime spikes:** Some trials showed a significant runtime increase for an individual polynomial. This did not correlate with coefficient magnitude or polynomial size; its cause is unknown.
- **Scalability:** Horner's method scales well to high polynomial degrees, maintaining stable computation times compared with the classical algorithm.

2 Task 2

2.1 Overview of Neville's algorithm

Neville's algorithm evaluates an interpolating polynomial at a given point. It efficiently evaluates polynomials from a specified set of points, although its $O(n^2)$ complexity can be costly for large datasets.

Neville interpolation uses a recurrence to compute the function value at x from points (x_i, y_i). Tests sampled n points from $D = (0, 10)$, then compared interpolated and actual function values at 100 evenly spaced points.

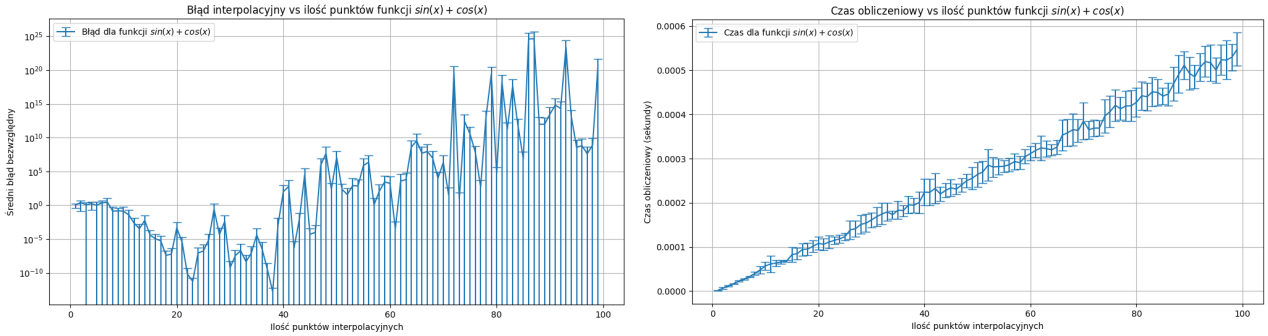
Four functions were tested:

- Trigonometric function: $f(x) = \sin(x) + \cos(x)$,
- Degree-3 polynomial: $f(x) = x^3 - 2x + 1$,
- Exponential function: $f(x) = e^x$,
- Rational function: $f(x) = \frac{1}{1+x^2}$.

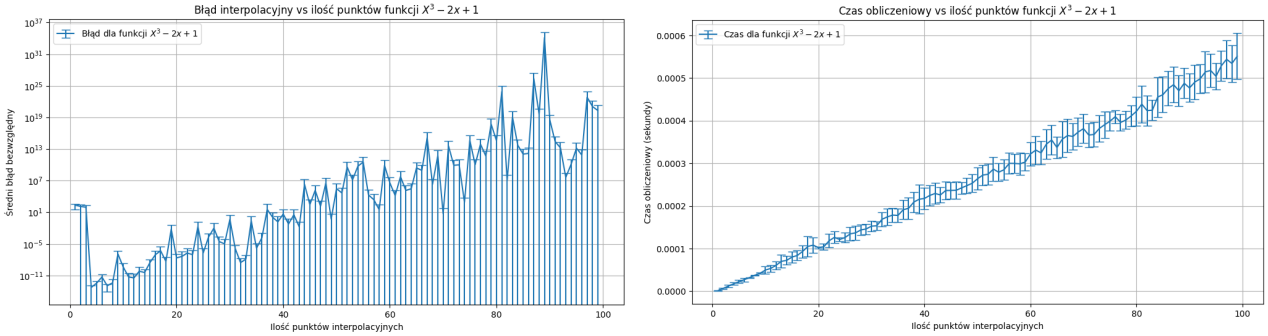
For each function, the interpolation error, its standard deviation and the algorithm runtime were computed.

2.2 Results and conclusions

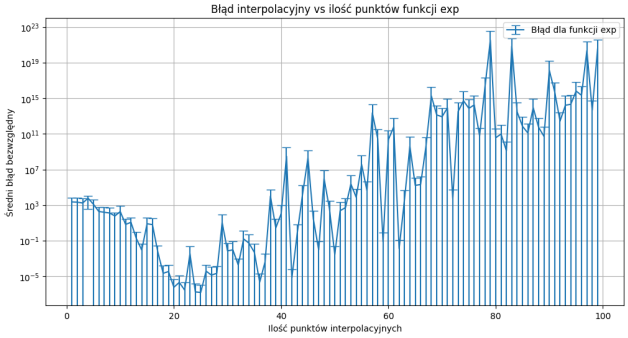
Trigonometric-function results: Minimum error: $2.01 \times 10^{-13} \pm 1.03 \times 10^{-12}$, at 23 points.

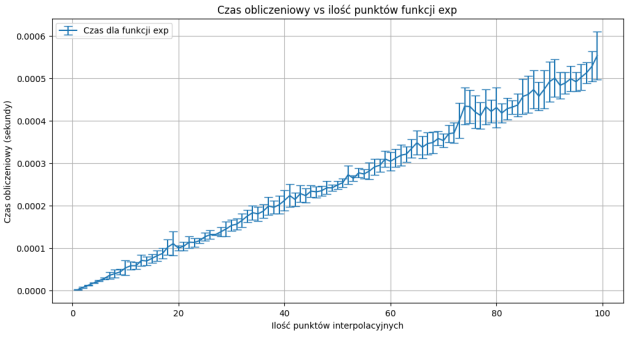


Degree-3 polynomial results: Minimum error: $8.21 \times 10^{-14} \pm 1.31 \times 10^{-13}$, at 9 points.

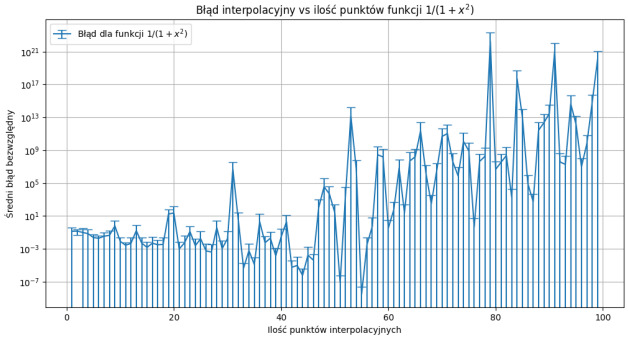


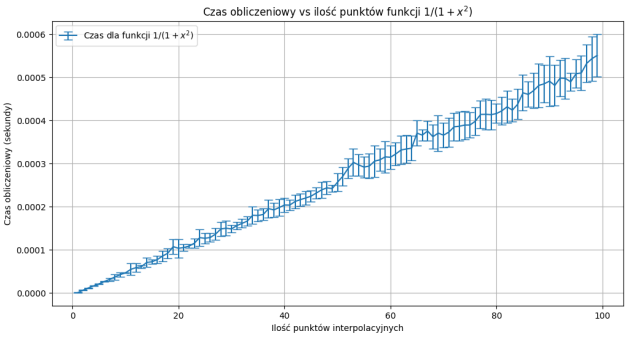
Exponential-function results: Minimum error: $2.00 \times 10^{-9} \pm 1.25 \times 10^{-8}$, at 26 points.





Results for $\frac{1}{1+x^2}$: Minimum error: $7.63 \times 10^{-12} \pm 3.28 \times 10^{-11}$, at 28 points.





Conclusions:

- **Precision for 10–20 points:** Using 10 to 20 points gave the best accuracy. More points increased errors, mainly through numerical effects from large increments. However, the standard deviation was large.
- **Computation time:** Neville's runtime was linear in the number of points, with a very small growth coefficient. This follows from the implementation.
- **Differences between functions:** The exponential function gave the largest errors because of its rapid growth over D. The cubic polynomial gave the smallest errors, as expected because polynomial interpolation is ideal for this function type.
- **Standard deviation and error:** The unusual display of standard deviation is caused by plot rescaling. It mainly indicates whether the deviation rises substantially.

3 Task 3/4

3.1 Overview of Newton's algorithm

Newton's divided-difference algorithm interpolates functions. Its main advantage is efficient evaluation of an interpolating polynomial from a given point set. Its $O(n^2)$ complexity makes it most suitable for moderate input sizes.

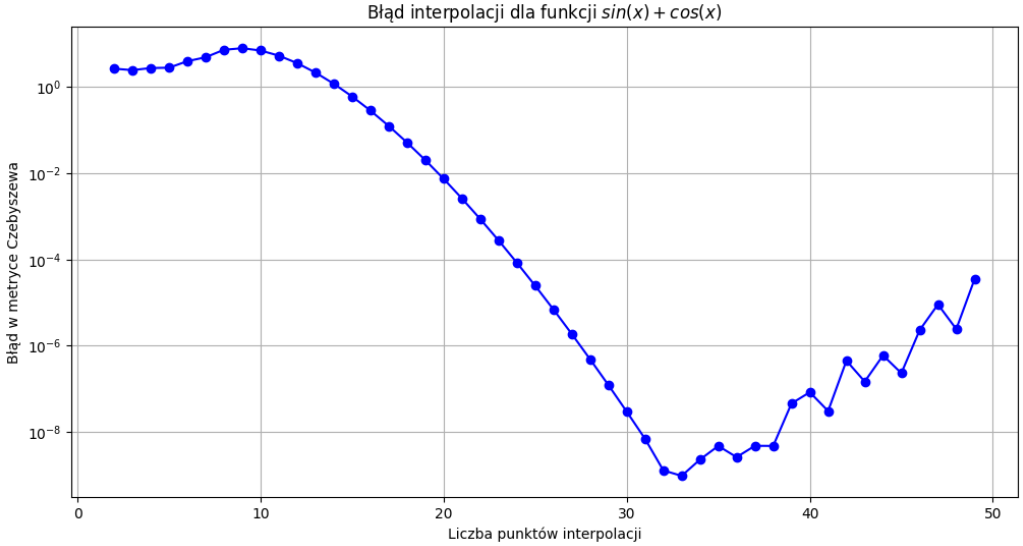
Divided differences allow terms to be added incrementally without recomputing the entire polynomial. Tests examined Chebyshev-metric errors at 1000 points with a random sample of n points, assessing how well the fit reproduces the function.

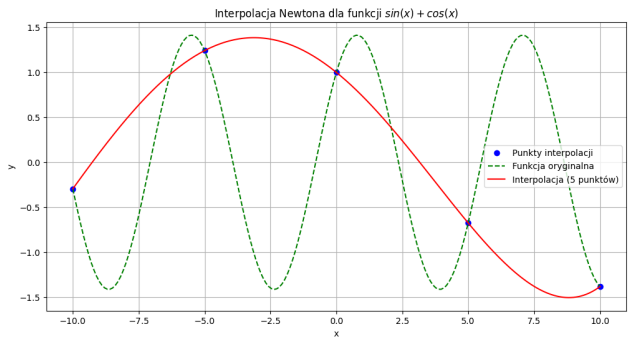
Three functions were tested:

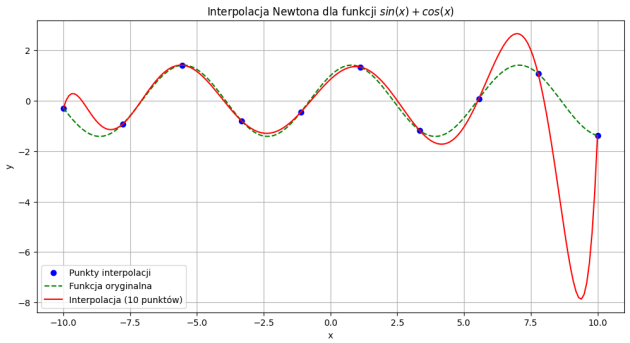
- Trigonometric function: $f(x) = \sin(x) + \cos(x)$,
- Rational function: $f(x) = \frac{1}{1+x^2}$,
- Discontinuous function: $f(x) = \lfloor x \rfloor$.

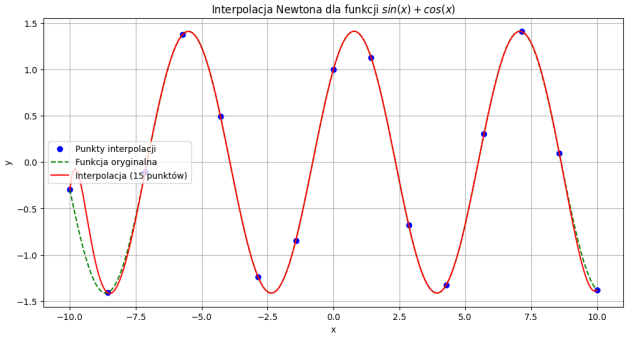
3.2 Results and conclusions

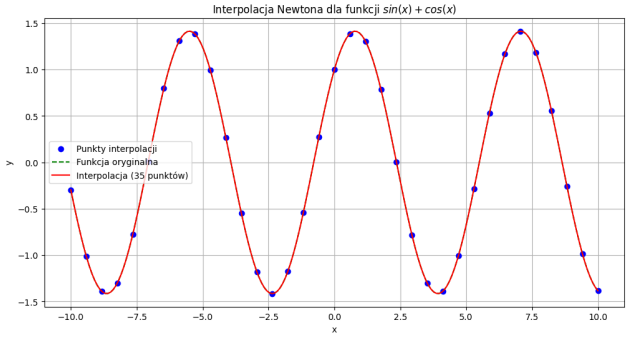
Results for $\sin(x) + \cos(x)$**:** Minimum error: 9.63×10^{-10}, achieved with 33 points.



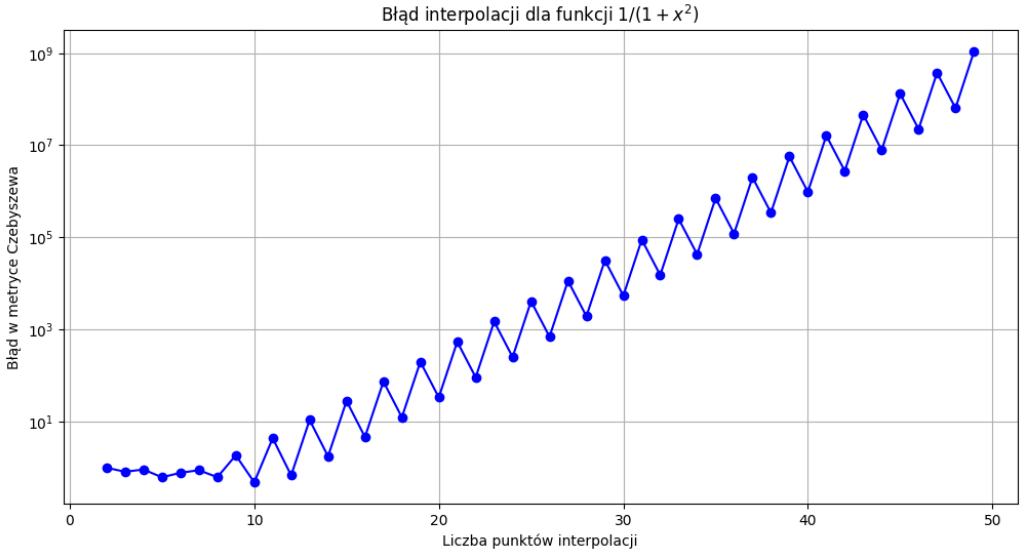


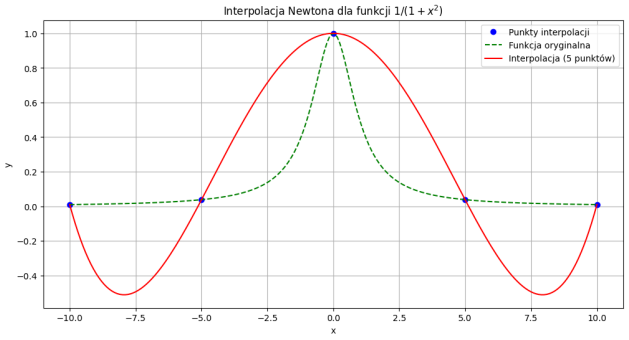


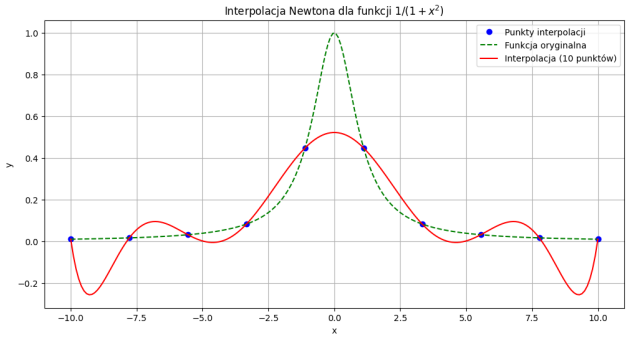


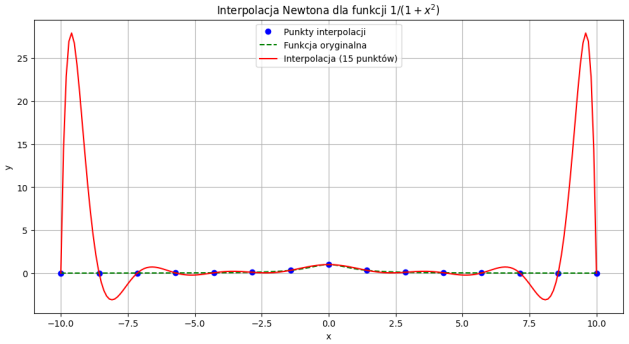


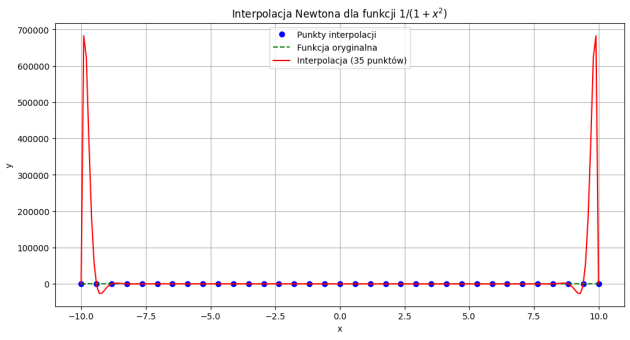
Results for $\frac{1}{1+x^2}$: Minimum error: 4.78×10^{-1}, achieved with 10 points.



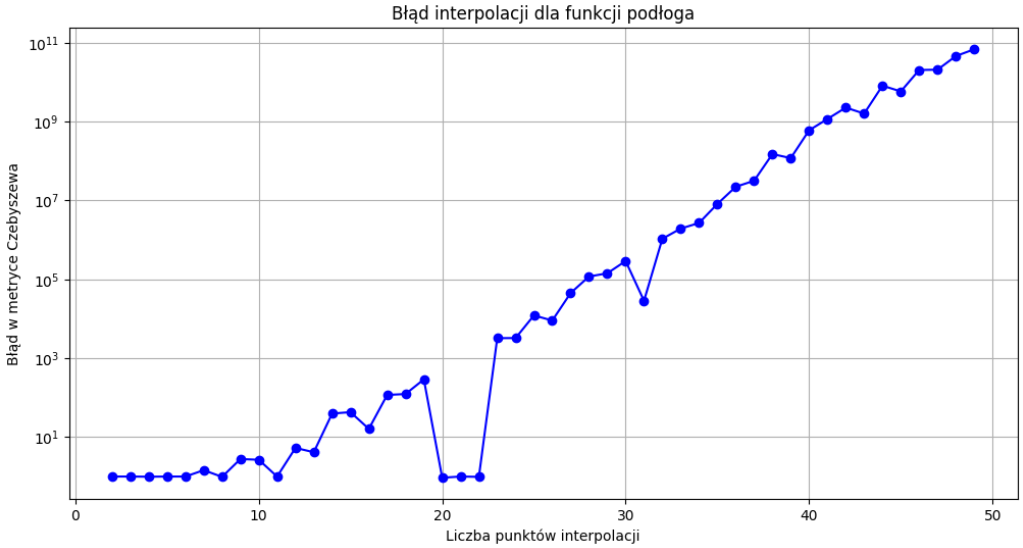


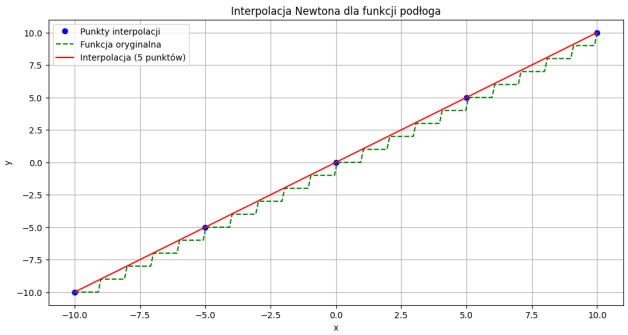


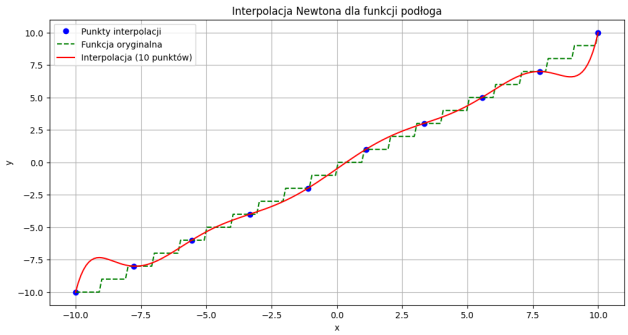


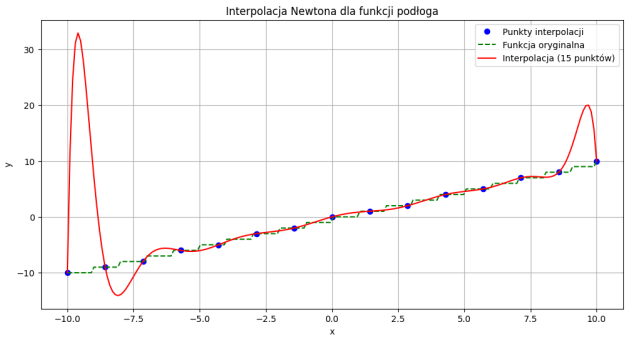


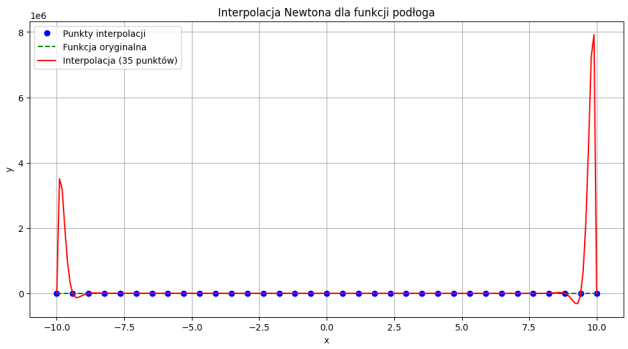
Results for $\lfloor x \rfloor$: Minimum error: 9.32×10^{-1}, achieved with 20 points.











Conclusions:

- **Efficiency with a moderate number of points:** Adding many interpolation points does not necessarily improve results. It can instead increase errors as the algorithm fits a polynomial requiring abrupt changes in function values.
- **Difficulty with discontinuous functions:** Discontinuous functions, such as $\lfloor x \rfloor$, produce much larger errors because a continuous polynomial cannot fit discontinuities well.
- **Uniformly continuous functions:** Trigonometric and rational functions give better results, although accuracy limitations remain.
- Functions with many extrema show characteristic jumps caused by frequent changes in their rate of growth.

4 Task 5

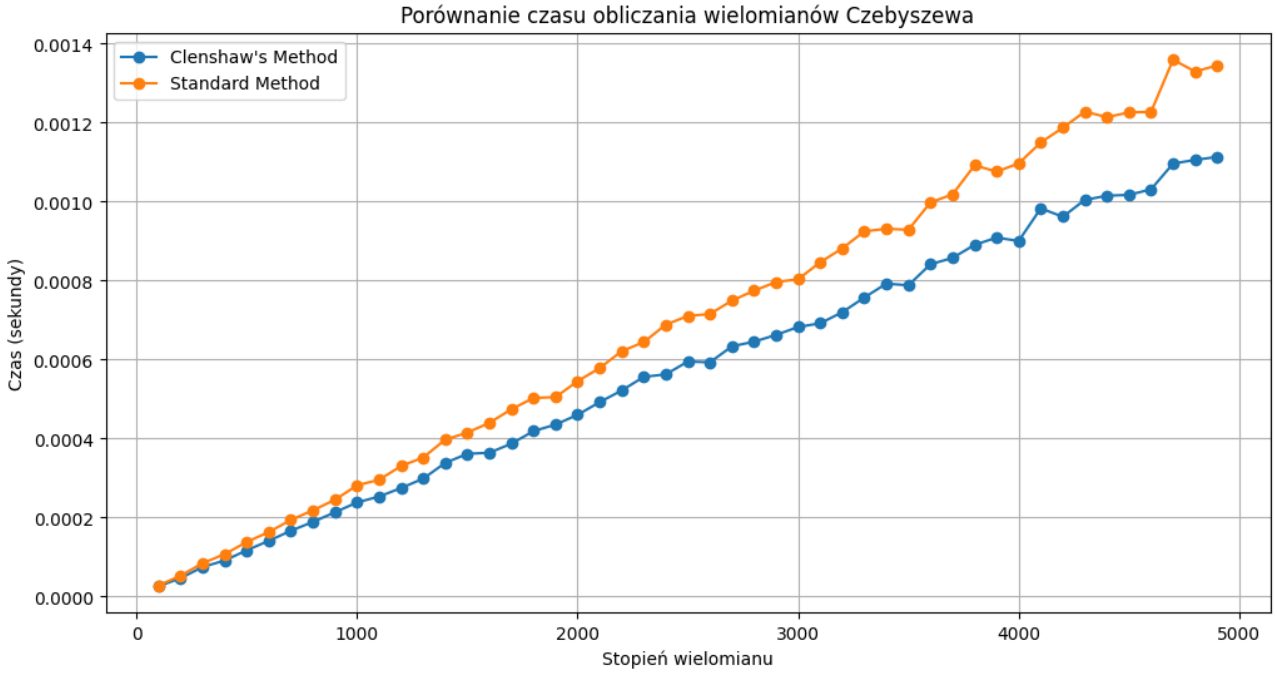
4.1 Overview of Clenshaw's algorithm

Clenshaw's algorithm evaluates a polynomial in Chebyshev form with $O(n)$ complexity. Compared with standard polynomial evaluation, its recursive treatment significantly accelerates computation.

The experiment compared Clenshaw's algorithm with classical Chebyshev-polynomial evaluation. The plot shows mean runtimes for both methods. Settings were as in Task 1.

4.2 Results and conclusions

Mean time for Clenshaw's method: 0.000578 ± 0.000324 **Mean time for the standard method:** 0.000692 ± 0.000393 **Speedup:** $1.20 \times \pm 0.03\times$



Conclusions:

- **Speed of Clenshaw's method:** Clenshaw's algorithm clearly outperforms standard Chebyshev-polynomial evaluation, running an average of 1.20x faster.
- **Comparison with Horner:** Clenshaw's method approaches the optimal runtime known from Horner's algorithm, which is a special case for Chebyshev polynomials.
- **Runtime stability:** Unlike Horner's algorithm, Clenshaw's method does not exhibit runtime fluctuations, indicating greater numerical stability.

5 Task 6

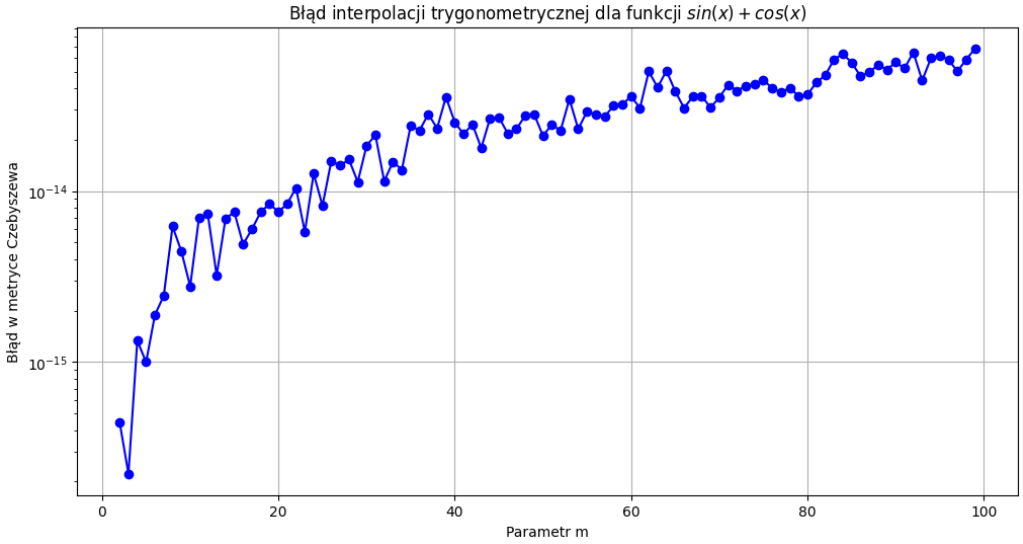
5.1 Overview of trigonometric interpolation

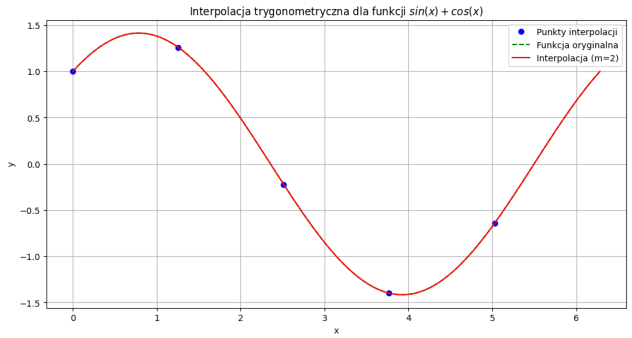
Trigonometric interpolation approximates periodic functions with trigonometric polynomials. Its complexity is $O(n \cdot m)$, where n is the number of interpolation points ($n = 2m$) and m is the number of polynomial coefficients. It uses periodic function values at evenly spaced nodes to determine the approximation coefficients.

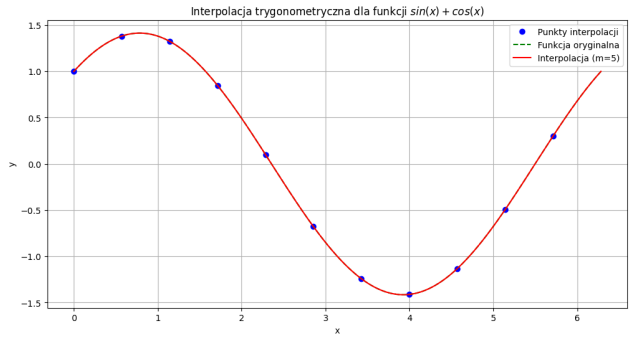
The experiment examined how interpolation error changes with the number of points m, and how well the algorithm reproduces selected continuous and discontinuous functions. Results are plotted below. Settings were as in Task 3/4.

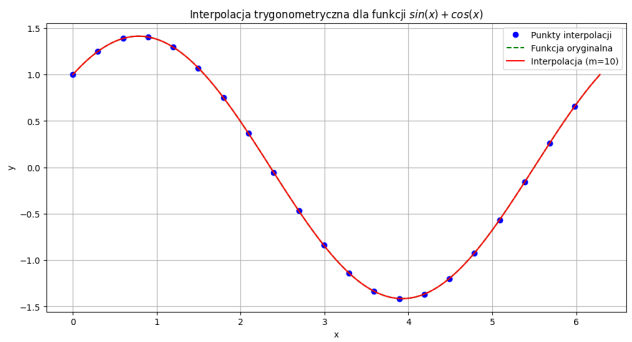
5.2 Results and conclusions

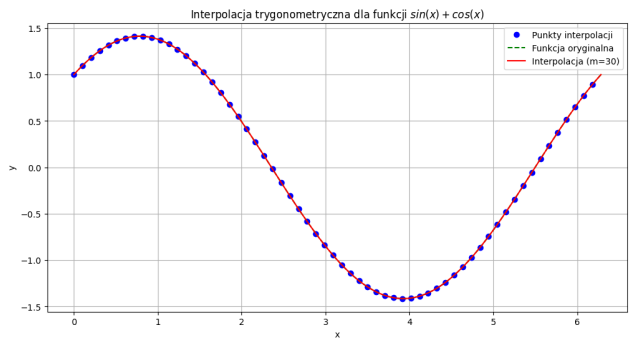
Results for $\sin(x) + \cos(x)$**:** Minimum error: $2.22e-16$, achieved for $m = 3$.



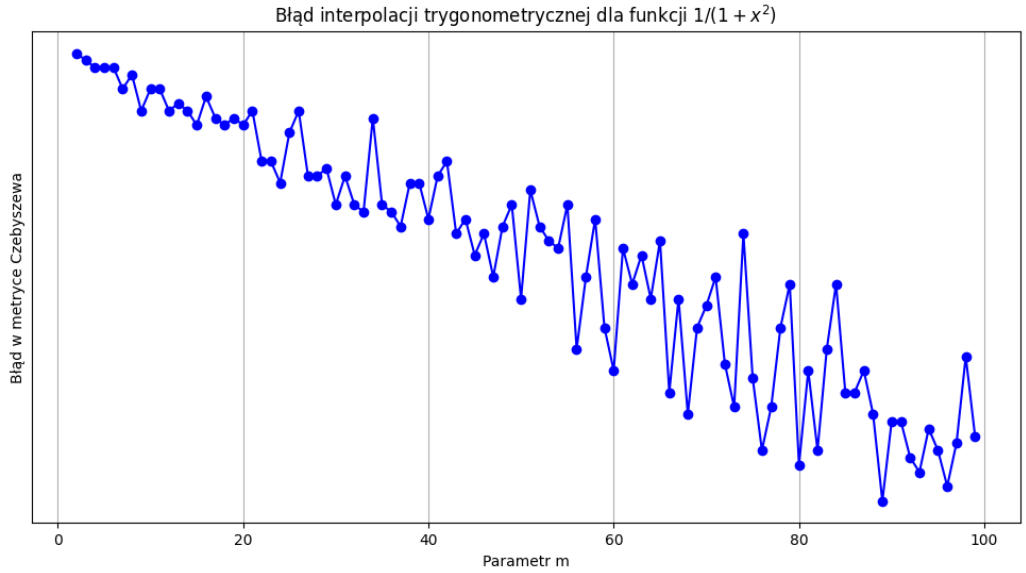


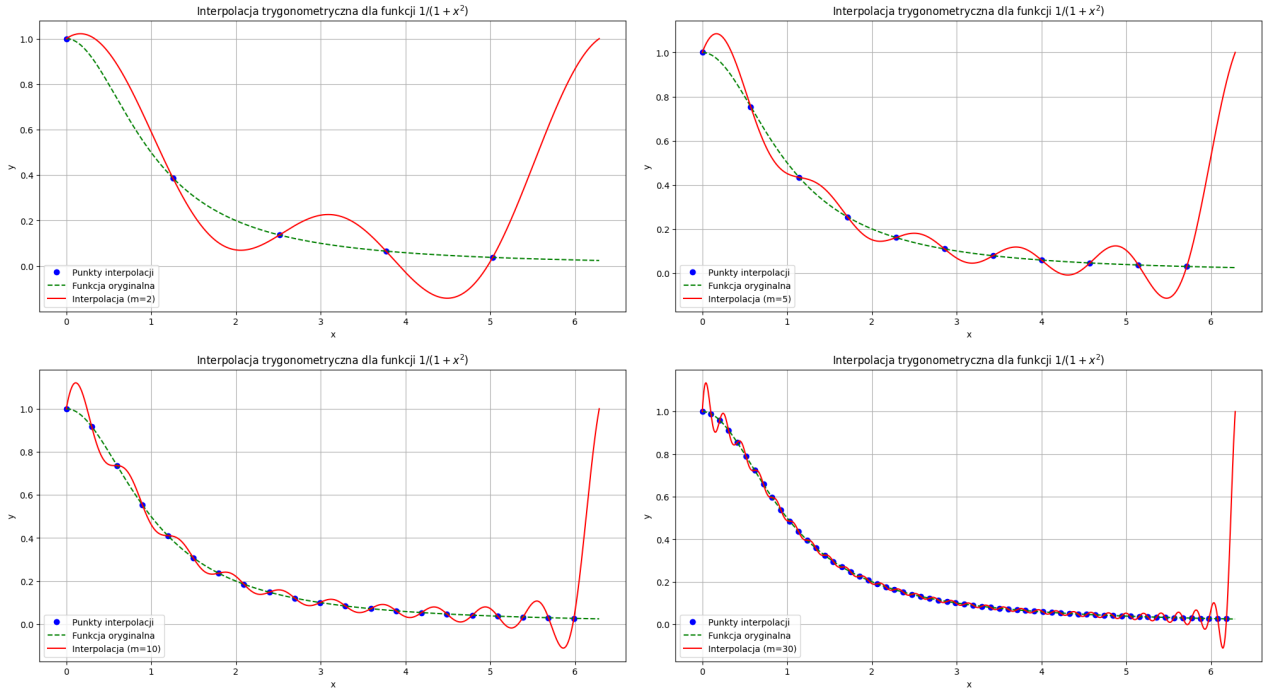




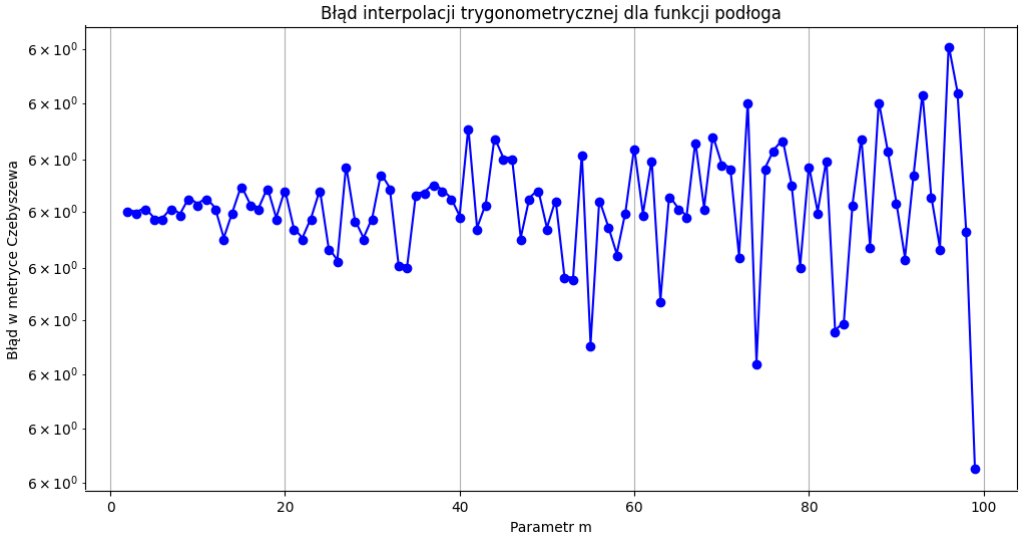


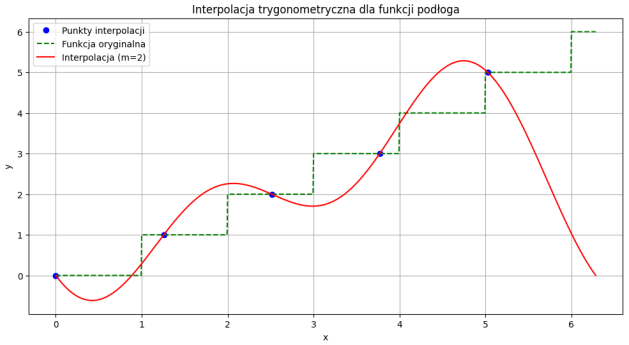
Results for $\frac{1}{1+x^2}$: Minimum error: $9.75e-01$, achieved for $m = 89$.

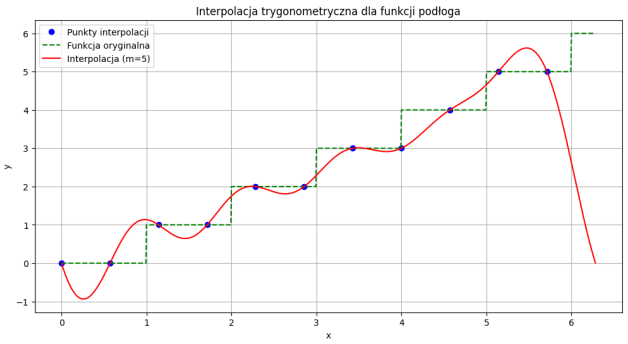


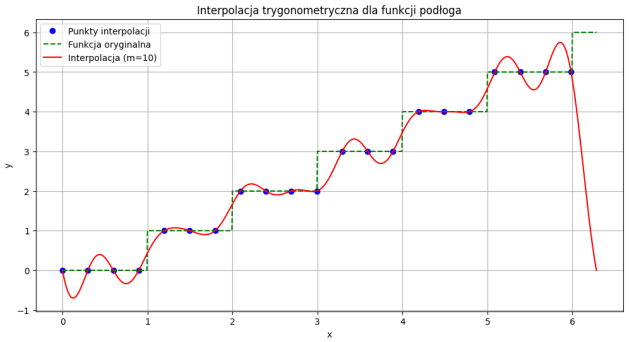


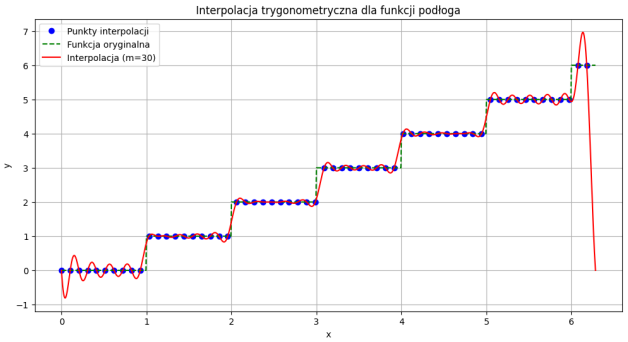
Floor-function results: Minimum error: $6.00e+00$, achieved for $m = 99$.











Conclusions:

- **Accuracy:** Trigonometric interpolation gives very small errors for periodic functions such as $\sin(x) + \cos(x)$.
- **Discontinuous functions:** The algorithm can interpolate discontinuous functions, such as the floor function, although their errors are much larger.
- **Endpoint deviations:** Fluctuations at the ends of the error plots arise from implementation imperfections that could be corrected. In theory, the method should produce the expected results.
- **General applicability:** Scaling the domain to $[0, 2\pi]$ makes the method applicable to many function types.